

HoneyPot Sensor Lab Technical Documentation

PHASE 2: THE SENTINEL – ACTIVE DEFENSE & SIEM INTEGRATION



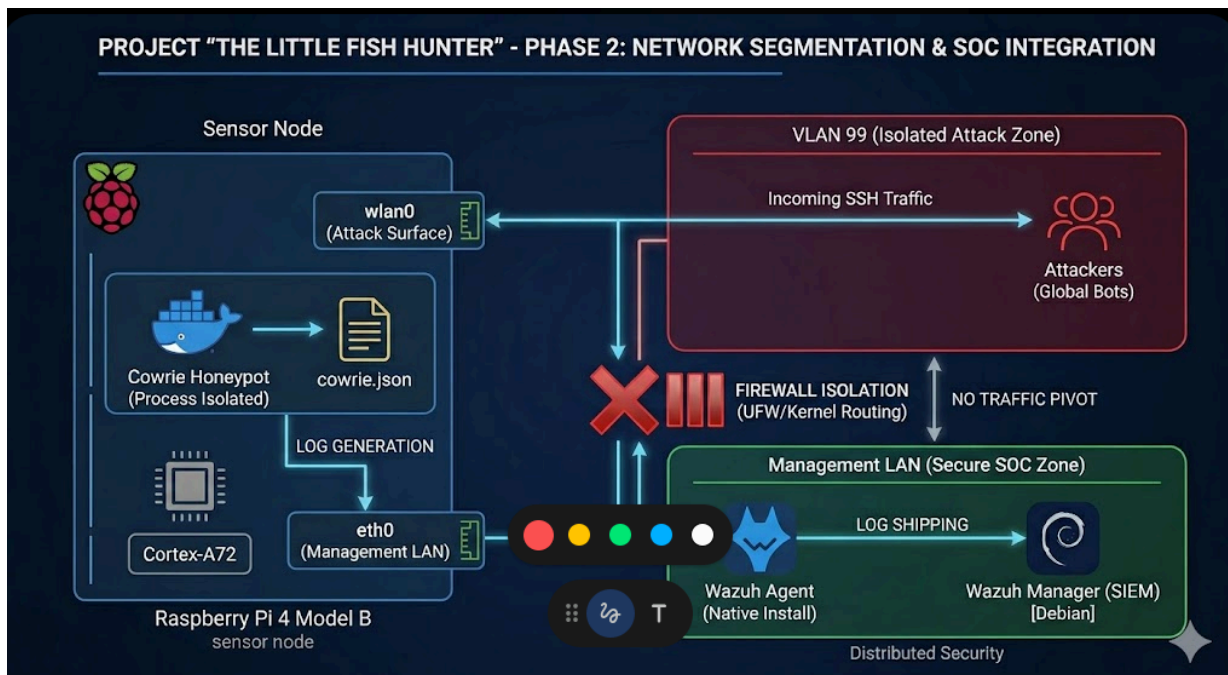
- **Project Lead:** Jacek Kaplon
- **Administrator:** Cybersecurity Enthusiast | IT Specialist
- **Status:** Phase 2 Complete – LAN Validated
- **Release Date:** April 7, 2026
- **Platform:** Raspberry Pi 4 (Agent) + Debian x86 (Wazuh Manager)

1. Executive Summary

This document describes the architecture, configuration, and validation of a distributed honeypot monitoring system built on commodity hardware. The system uses a Raspberry Pi 4 as a network sensor running the Cowrie SSH honeypot inside Docker, integrated with a Wazuh SIEM manager for real-time alerting and threat analysis. Phase 2 validates the complete stack in an isolated LAN.

2. Network Architecture & Infrastructure

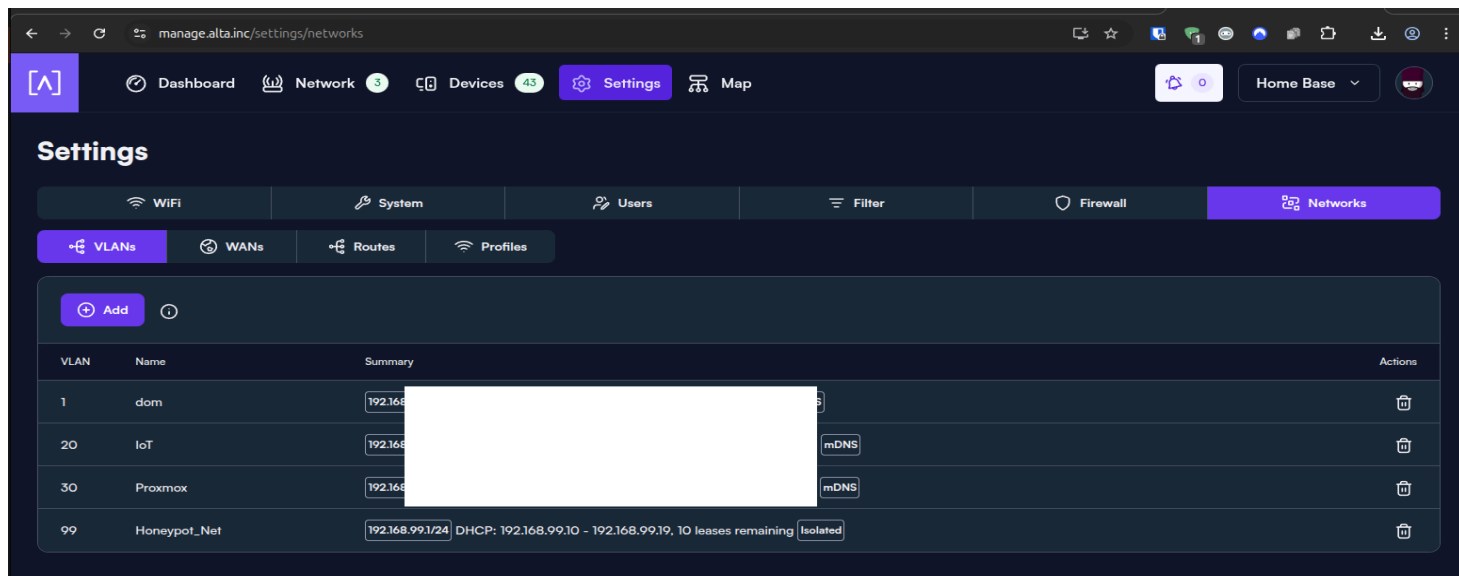
To ensure the safety of the Core SOC environment, strict isolation was implemented using the **Alta Labs Route10** router and kernel-level routing.



- **Attack Zone (VLAN 99):** An isolated Wi-Fi segment where the sensor's **wlan0** interface listens for incoming threats.
- **Management Zone (LAN):** The **eth0** interface connects directly to the Wazuh Manager (192.168.1.22) for secure log shipping.
- **Zero-Trust Isolation:** Routing rules ensure that traffic from the attack surface cannot reach the management network.

2.1 Network Topology

- **VLAN 99 (Attack Surface):** Isolated WiFi network connected via **wlan0**.
- **Management LAN (SOC):** Connected via **eth0** to the Wazuh Manager (**192.168.1.22**).



VLAN 99 - Isolation on the Router

2.2 Routing

Custom routing rules ensure that traffic arriving on **wlan0** cannot pivot to the internal LAN. The honeypot is exposed, but the management plane remains invisible.

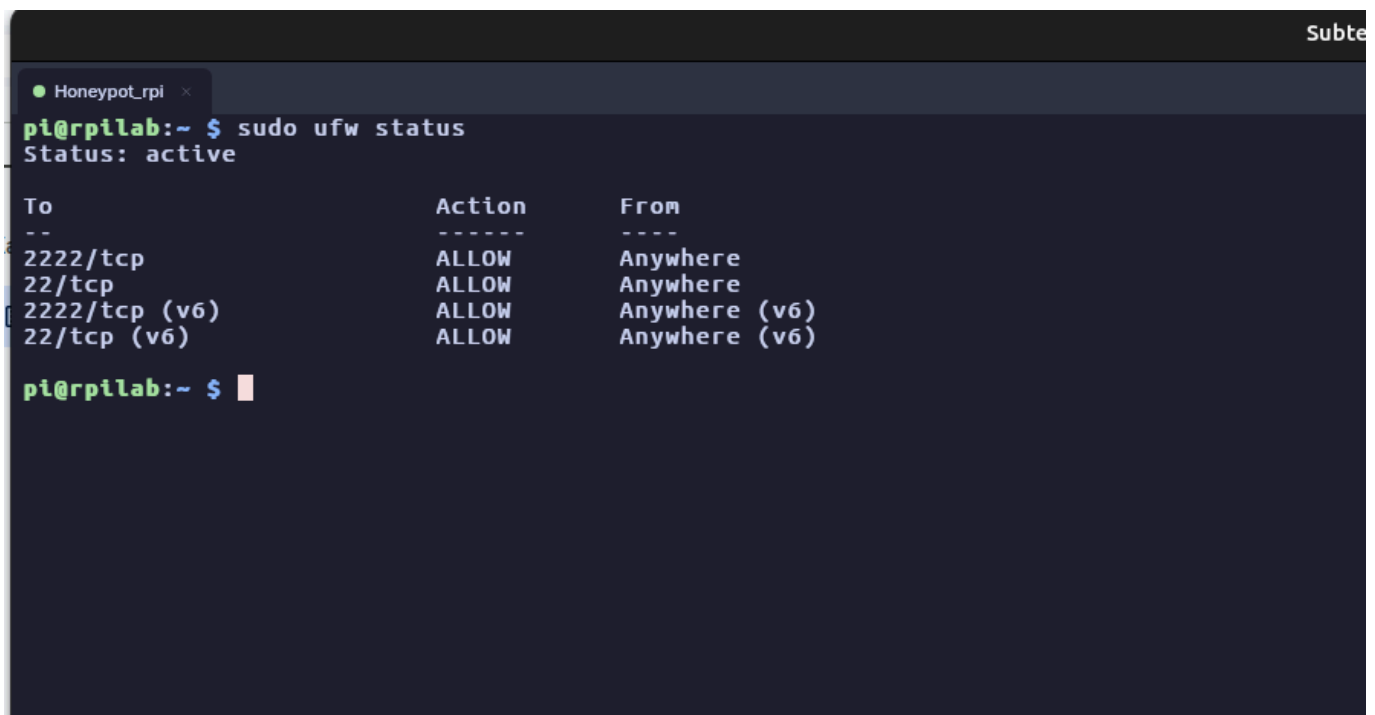
```
Subterm
Honeypot_rpi
pi@rpiLab:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
   link/ether e4:5f:01:49:e7:83 brd ff:ff:ff:ff:ff:ff
   inet 192.168.1.11/24 brd 192.168.1.255 scope global noprefixroute eth0
       valid_lft forever preferred_lft forever
   inet6 fe80::e65f:0149:e783/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether e4:5f:01:49:e7:84 brd ff:ff:ff:ff:ff:ff
   inet 192.168.99.10/24 brd 192.168.99.255 scope global noprefixroute wlan0
       valid_lft forever preferred_lft forever
   inet6 fe80::a694:b2b0:3cc2:fad8/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
4: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
   link/ether 2a:5b:bd:10:01:00 brd ff:ff:ff:ff:ff:ff
   inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
   inet6 fe80::285b:bdff:fe10:160/64 scope link proto kernel ll
       valid_lft forever preferred_lft forever
5: vethbdsce15@f2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default
   link/ether da:4c:9c:cd:fb:53 brd ff:ff:ff:ff:ff:ff link-netnsid 0
   inet6 fe80::d84c:9cfd:fb53/64 scope link proto kernel ll
       valid_lft forever preferred_lft forever
pi@rpiLab:~$ ip route show
default via 192.168.99.1 dev wlan0 proto static metric 10
default via 192.168.1.1 dev eth0 proto static metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.11 metric 100
192.168.1.157 via 192.168.1.1 dev eth0 proto static metric 100
192.168.1.158 via 192.168.1.1 dev eth0 proto static metric 100
192.168.99.0/24 dev wlan0 proto kernel scope link src 192.168.99.10 metric 10
pi@rpiLab:~$
```

ip addr show / ip route show - default gateway

2.3 Host Hardening (Raspberry Pi)

The host OS is fortified to prevent a honeypot compromise from affecting the management network.

- Admin Access: Port 2222/tcp, SSH Public Key only (ed25519), Passwords disabled.
- Firewall (UFW): Default DENY policy for all inbound traffic

A terminal window titled 'HoneyPot_rpi' showing the output of the 'sudo ufw status' command. The output indicates that the firewall is active and lists four rules: two for port 22 and two for port 2222, all allowing traffic from anywhere.

```
pi@rpilab:~ $ sudo ufw status
Status: active

To Action From
--
2222/tcp ALLOW Anywhere
22/tcp ALLOW Anywhere
2222/tcp (v6) ALLOW Anywhere (v6)
22/tcp (v6) ALLOW Anywhere (v6)

pi@rpilab:~ $
```

`sudo ufw status` - "Active" ruller 22 (Allow) i 2222 (Allow).

3 Honeypot Implementation (Docker / Cowrie)

The honeypot is containerized to provide filesystem isolation. Cowrie emulates a vulnerable SSH server and records every interaction.

- **Image:** [cowrie/cowrie](#) (Docker Hub)
- **Port Mapping:** Host 22 -> Container 2222
- **Log Format:** Structured JSON

```
Subterm
HoneyPot_rpl
pi@rpi1ab:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
51c8159cac4d   cowrie/cowrie:latest  "/cowrie/cowrie-env/_"  8 hours ago   Up 8 hours   2223/tcp, 0.0.0.0:22->2222/tcp, [::]:22->2222/tcp  cowrie
pi@rpi1ab:~$
```

`docker ps` - "cowrie" container

4 SIEM Integration (Wazuh)

```
1 xxx@debian: ~
root@debian:/home/xxx# cat /var/ossec/etc/rules/local_rules.xml
<group name="local,syslog,sshd,geoip,">

  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
    <srcip>1.1.1.1</srcip>
    <description>sshd: authentication failed from IP 1.1.1.1.</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>

  <rule id="100101" level="3">
    <decoded_as>json</decoded_as>
    <field name="clientIp">\.+\</field>
    <description>Technitium DNS: Log zlokalizowany przez srcip</description>
    <group>geoip,</group>
  </rule>

  <rule id="100201" level="5">
    <decoded_as>json</decoded_as>
    <field name="eventid">cowrie.login.failed</field>
    <description>Cowrie: Nieudane logowanie do Honeypota z adresu ${src_ip}</description>
    <group>cowrie,authentication_failed,</group>
  </rule>

  <rule id="100202" level="7">
    <decoded_as>json</decoded_as>
    <field name="eventid">cowrie.login.success</field>
    <description>Cowrie: SUKCES logowania do Honeypota (Haker wszedł!) z adresu ${src_ip}</description>
    <group>cowrie,authentication_success,</group>
  </rule>
</group>
root@debian:/home/xxx#
```

Custom rules in Wazuh

Custom Rules (Manager Side):

- **Rule 100201 (Level 5):** Failed honeypot login attempt.
- **Rule 100202 (Level 7):** Successful honeypot login (Critical alert).

6. System Validation (LAN Demo)

Validation was performed using an Ubuntu machine (192.168.1.12) to simulate brute-force attacks.

The screenshot displays the Wazuh Manager interface with a table of alerts and a terminal window showing the source of the attack.

Alert Table:

Field	Value
@timestamp	Apr 7, 2026 @ 21:42:58.251
_index	wazuh-alerts-4.x-2026.04.07
agent.id	017
agent.ip	192.168.1.11
agent.name	rpilab
data.eventid	cowrie.login.failed
data.message	login attempt [pi/admin123] failed
data.password	admin123
data.protocol	ssh
data.sensor	51c8159cac4d
data.session	2bdbb0026504
data.src_ip	192.168.1.12
data.timestamp	Apr 7, 2026 @ 21:37:10.592
data.username	pi
data.uuid	32df9ab2-2fe7-11f1-a9b7-da8176cc5bca
decoder.name	json
full_log	{ "eventid": "cowrie.login.failed", "username": "pi", "password": "admin123", "message": "login attempt [pi/admin123] failed", "sensor": "51c8159cac4d", "uuid": "32df9ab2-2fe7-11f1-a9b7-da8176cc5bca", "timestamp": "2026-04-07T20:37:10.592Z", "src_ip": "192.168.1.12", "session": "2bdbb0026504", "protocol": "ssh" }
id	1775594578.23847595
input.type	log
location	/home/pi/cowrie/var/log/cowrie/cowrie.json
manager.name	debian
rule.description	Cowrie: Nieudane logowanie do Honeypota z adresu 192.168.1.12
rule.firedtimes	13
rule.groups	local, syslog, sshd, geoup, cowrie, authentication_failed
rule.id	100201
rule.level	5
rule.mail	false
timestamp	Apr 7, 2026 @ 21:42:58.251

Terminal Window (HoneyPot_rpi):

```
pi@rpilab:~$ tail -n 1 -/cowrie/var/log/cowrie/cowrie.json | python3 -m json.tool
{
  "eventid": "cowrie.session.closed",
  "duration": "120.1",
  "message": "Connection lost after 120.1 seconds",
  "sensor": "51c8159cac4d",
  "uuid": "32df9ab2-2fe7-11f1-a9b7-da8176cc5bca",
  "timestamp": "2026-04-07T20:38:54.036604Z",
  "src_ip": "192.168.1.12",
  "session": "2bdbb0026504",
  "protocol": "ssh"
}
```

End-to-End Alert Validation. The composite view shows a successful telemetry pipeline: a brute-force attempt captured in the `cowrie.json` log (top-right) is instantly processed by the Wazuh Manager, triggering Rule 100201 with full visibility of the attacker's credentials and source IP

7. Technical Summary & Phase 2 Conclusion

System Infrastructure:

- **OS / Platform:** Raspberry Pi OS (Debian 12 Bookworm).
- **Admin Access:** Port 2222/TCP, SSH Key (ed25519) only, Password Authentication disabled.
- **Honeypot Engine:** Cowrie SSH (deployed via Docker) on Port 22/tcp.

SIEM & Detection Logic:

- **Integration:** Wazuh Agent (Native) connected to centralized Wazuh Manager (Debian).
- **Log Pipeline:** Real-time JSON telemetry ingestion from </home/pi/cowrie/var/log/cowrie/cowrie.json>.
- **Custom Rules:** Implemented in [local_rules.xml](#) on the Manager side:
 - **Rule 100201 (Level 5):** Failed honeypot login attempt (Brute-force detection).
 - **Rule 100202 (Level 7):** Successful honeypot login (Critical compromise alert).

Network Security & Hygiene:

- **Segmentation:** Dual-interface architecture using VLAN 99 ([wlan0](#)) for the attack surface and Management LAN ([eth0](#)) for SOC connectivity.
- **Hardware:** Routed via Alta Labs Route10 with strict isolation policies.
- **Security Analysis:** The containerized approach (Docker) ensures that potential compromises are sandboxed, preventing lateral movement to the Core SOC infrastructure.

Status: PHASE 2 COMPLETE - FULLY OPERATIONAL

